CICLO FORMATIVO DE GRADO SUPERIOR · DESARROLLO DE APLICACIONES WEB

Casino DAW · Guía de integración

Login, sesión y saldo · Proyecto transversal DWEC · DWES · 2º DAW / 2º DAM · Curso 2026-2027 · Profesor: Alberto Sanchis

Cómo usar esta guía

¡Hola a todos! En esta guía os explico, paso a paso, el código PHP que he subido a mi rama `Alberto` (login, sesión, landing y panel de control) para que podáis conectar vuestro juego en JavaScript con él.

> La regla de oro: **no inventéis nombres**. Las variables de sesión, las columnas de la base de datos y los nombres de archivo son los que yo uso; si usáis otros, vuestro juego no encajará.

Cada paso de la guía es una actividad (**CA.1** a **CA.7**) y sigue el mismo esquema:

BLOQUE	QUÉ ES
Teoría	Qué hace cada archivo y por qué está hecho así.
Pregunta para pensar	Antes de seguir leyendo, intentad responderla en voz alta o en parejas.
Ejercicio	Algo que tenéis que hacer en vuestro ordenador.
Reto «¿dónde está el fallo?»	Mi código funciona, pero no es perfecto, y algunas cosas las he dejado así a propósito. Vuestra misión es encontrar qué podría fallar o hacerse mejor.

Cómo se trabaja

- **No toquéis nada en mi rama.** Apuntad lo que encontréis y lo comentamos en clase.
- Cada actividad tiene su enunciado en **Aplica** (y en Teams). Esta guía es el mapa completo.
- Material: XAMPP (Apache + MySQL) arrancado, phpMyAdmin, un editor y **la consola del navegador (F12) siempre abierta**.
- Parejas. La última página es la **Chuleta de nombres**: tenedla delante siempre.

> *Antes de empezar, pregunta para todo el grupo: vuestro juego ahora mismo guarda el saldo en una variable de JavaScript.* ***¿Qué pasa con ese saldo si pulsáis F5?*** *¿Y si otro compañero abre vuestro juego en su ordenador? Esa es la razón de todo lo que viene después.*

CA.1 El mapa del casino

ORDENADOR · PAREJA · 50 MIN

Mi casino son 8 archivos en la raíz de mi rama `Alberto`. Vuestro juego será una página más dentro de este mapa, así que lo primero es saber qué hace cada pieza.

ARCHIVO	QUÉ HACE	¿LO TOCÁIS VOSOTROS?
`casino.sql`	Crea la base de datos `casino` y la tabla `usuarios` con 2 usuarios de prueba	Solo lo importáis
`conexion.php`	Abre la conexión a MySQL y la deja en la variable `$conexion`	Sí, en vuestra copia local (CA.3)
`index.php`	Formulario de login: teléfono y contraseña	No
`usuario.php`	Comprueba las credenciales y crea la sesión	No
`sesion.php`	El «portero»: si no hay sesión, te devuelve a `index.php`	No, pero lo incluiréis en vuestro juego
`juegos.php`	Landing: «Bienvenido nombre», el saldo y los enlaces a los juegos	No
`panelcontrol.php`	Panel del admin: crear usuarios, añadir saldo y borrar	No
`logout.php`	Cierra la sesión	No

El recorrido: `index.php` (formulario) → POST a `usuario.php` → si eres admin, `panelcontrol.php`; si eres jugador, `juegos.php`; si fallas, vuelta a `index.php?error=1`. Desde `juegos.php` se va a cada juego o a `logout.php`, que te devuelve al login. `juegos.php`, `panelcontrol.php` y vuestro juego empiezan con `require "sesion.php"`: sin sesión, vuelven a `index.php`.

PREGUNTA PARA PENSAR

`usuario.php` no tiene ni una línea de HTML. Entonces, ¿qué ve el usuario mientras se ejecuta? ¿Cómo llega a la siguiente página?

EJERCICIO 1.1 Recorre el mapa a mano

Abrid cada archivo y apuntad en una tabla a qué otro archivo manda y cómo lo hace: `header("Location: ...")`, un `href`, un `action` de formulario o un `require`. Cuando acabéis, comparad vuestra tabla con el recorrido de arriba. ¿Falta alguna flecha?

RETO 1.2 ¿Dónde está el fallo?

`juegos.php` enlaza con cuatro juegos. Buscad esos archivos en la rama. ¿Existen? ¿Qué pasará cuando un jugador pulse en uno? Y si vuestro juego es de dados, blackjack o pares o nones, ¿qué tendría que cambiar para que aparezca en la landing?

RETO 1.3 Lo que no se ve

Todas las páginas cargan `css/estilos.css`. Abrid la pestaña Red (Network) de las herramientas del navegador y recargad. ¿Qué código de respuesta sale para ese archivo? ¿Por qué la página funciona igualmente?

RETO 1.4 **La documentación también miente**

En la carpeta `doc_tecnica` escribí, antes de programar, una arquitectura con carpetas `includes/`, `admin/` y archivos como `login.php` o `landing.php`. Comparadla con los archivos reales y apuntad al menos tres diferencias. ¿Cuál manda, la documentación o el código? ¿Por qué es peligroso que no coincidan?

ENTREGA — la tabla del recorrido (archivo → destino → cómo) y la respuesta a los tres retos.

CA.2 La base de datos

ORDENADOR · PAREJA · 50 MIN

Todo el casino vive en una sola tabla: `usuarios`, dentro de la base de datos `casino`. Vuestro juego solo necesitará leer y cambiar una columna, `saldo`, pero tenéis que conocerlas todas para no inventar nombres.

COLUMNA	TIPO	PARA QUÉ SIRVE
`id`	INT, clave primaria, autoincremental	Identifica al usuario. Es el dato que se guarda en la sesión
`nombre`	VARCHAR(100)	El nombre del «Bienvenido …»
`telefono`	VARCHAR(20), UNIQUE	Es el «usuario» del login. No puede haber dos iguales
`password`	VARCHAR(255)	La contraseña **cifrada** con `password_hash()`, nunca en claro
`saldo`	DECIMAL(10,2), por defecto 0	El dinero del jugador. **Es la columna que os importa**
`rol`	ENUM('admin','jugador')	Decide si entras al panel o a los juegos

Usuarios de prueba que crea `casino.sql`:

TELÉFONO	CONTRASEÑA	ROL	SALDO
600000000	admin123	admin	0
611111111	1234	jugador	100

Teoría en dos ideas

- **¿Por qué DECIMAL y no FLOAT?** Los números con decimales en coma flotante no son exactos. Con dinero, un céntimo perdido en cada jugada acaba siendo un problema.
- **¿Por qué la contraseña es tan larga y rara?** Porque no se guarda la contraseña, sino un *hash*: una huella que no se puede «deshacer». Al hacer login no se descifra nada: se comprueba si lo que escribes produce una huella compatible.

PREGUNTA PARA PENSAR

El login pide el teléfono, no el nombre. ¿Por qué crees que se ha elegido una columna con UNIQUE para identificarse? ¿Qué pasaría si hubiera dos «Laura»?

EJERCICIO 2.1 **Monta la base de datos**

Abrid phpMyAdmin, pestaña Importar, y cargad `casino.sql`. Después, en la pestaña SQL, haced una consulta que muestre solo el nombre y el saldo de todos los usuarios. Apuntad la consulta en vuestro cuaderno.

EJERCICIO 2.2 **El saldo desde SQL**

Escribid la consulta que suma 50 € al jugador de prueba y otra que le resta 20 €. Fijaos en cómo lo hace `panelcontrol.php` (buscad `saldo = saldo +`). ¿Por qué es mejor eso que leer el saldo, sumarlo en PHP y volver a guardarlo?

RETO 2.3 **El usuario que no puede entrar**

Desde phpMyAdmin, insertad a mano un usuario nuevo escribiendo la contraseña tal cual (por ejemplo `hola`) en la columna `password`. Intentad entrar con él en `index.php`. ¿Por qué no funciona si la contraseña es correcta? ¿Cómo lo crearíais bien?

RETO 2.4 **Saldo en números rojos**

Ejecutad una consulta que le reste 500 € al jugador de prueba (que tiene 100). ¿Qué saldo queda? ¿Lo debería permitir la base de datos? Buscad en `doc_tecnica/02-base-de-datos.md` qué regla se suponía que iba a tener la columna `saldo` y comprobad si está en `casino.sql`.

RETO 2.5 **Cuidado al reimportar**

Mirad la línea de `casino.sql` que empieza por `DROP TABLE`. Si a mitad de curso volvéis a importar el archivo, ¿qué les pasa a los usuarios y saldos que habíais creado?

MINI RETO JS **0.1 + 0.2**

Escribid `0.1 + 0.2` en la consola del navegador. ¿Qué sale? Relacionadlo con la primera idea de teoría y pensad cómo afecta a vuestro saldo en JS.

ENTREGA — las consultas de 2.1 y 2.2, la respuesta razonada a los tres retos y el resultado del mini reto.

CA.3 La conexión con la base de datos

ORDENADOR · PAREJA · 30 MIN

`conexion.php` abre la puerta a MySQL y deja esa puerta abierta en una variable que se llama `$conexion`. Cualquier archivo que haga `require "conexion.php";` puede usar `$conexion` para hacer consultas. Vuestro juego también. Lo que hace, en cuatro pasos:

1. Guarda en cuatro variables los datos del servidor: `$servidor`, `$usuario`, `$clave` y `$bd`.
2. Llama a `mysqli_connect()` con esos cuatro datos.
3. Si la conexión falla, para todo con `die()` y muestra el error.
4. Fija la codificación en `utf8mb4` para que las tildes y la ñ lleguen bien.

PREGUNTA PARA PENSAR

Si cinco archivos necesitan la base de datos, ¿por qué es mejor tener un solo `conexion.php` que copiar esas líneas en los cinco? Pensad en qué pasa el día que cambia la contraseña del servidor.

EJERCICIO 3.1 **Tu conexión local**

En vuestro ordenador, con XAMPP, los datos son: servidor `localhost`, usuario `root`, contraseña vacía y base de datos `casino`. Haced vuestra copia local de `conexion.php` con esos datos y comprobad que `index.php` deja entrar al jugador de prueba.

EJERCICIO 3.2 **Provócalo**

Cambiad el nombre de la base de datos por uno que no exista y recargad. Apuntad el mensaje exacto que sale. Ahora pensad: ¿es buena idea que un jugador de verdad vea ese mensaje? ¿Qué información le estás dando?

RETO 3.3 **El comentario que no cuadra**

Leed el comentario que puse en la primera línea de `conexion.php` y luego los valores de las variables. ¿Dicen lo mismo? ¿Qué problema hay en que el código y su comentario no coincidan?

RETO 3.4 **Secretos en Git**

Un repositorio de GitHub lo puede leer mucha gente, y Git guarda todo el historial. Debatid en grupo:

- ¿Qué datos de `conexion.php` no deberían subirse nunca a un repositorio?
- Si alguien sube una contraseña y en el commit siguiente la borra, ¿ha desaparecido de verdad?
- ¿Para qué sirve el archivo `.gitignore`? ¿Y un `conexion.ejemplo.php` con los datos vacíos?

NORMA PARA EL PROYECTO

Vuestro `conexion.php` con datos reales **se queda en vuestro ordenador**. A Git solo sube una plantilla sin contraseñas.

ENTREGA — captura del login funcionando con vuestra conexión local, el mensaje de 3.2 copiado literalmente y las conclusiones del debate de 3.4.

CA.4 Login y variables de sesión

ORDENADOR · PAREJA · 50 MIN

Una sesión es una **«mochila»** que el servidor guarda para cada visitante. El navegador solo lleva una cookie con un número (`PHPSESSID`) y, con ese número, PHP sabe qué mochila es la tuya. Lo que metes en `$_SESSION` sigue ahí al cambiar de página, hasta que cierras la sesión.

El formulario (index.php)

Envía por POST a `usuario.php` dos campos: **`telefono`** y **`password`**. Si la URL trae `?error=1`, muestra «Teléfono o contraseña incorrectos». Y si ya hay sesión, ni siquiera enseña el formulario: te manda a `juegos.php`.

La comprobación (usuario.php)

1. Arranca la sesión con `session_start()`.
2. Busca en la tabla `usuarios` la fila con ese teléfono, usando una **consulta preparada** (el `?` de la SQL).
3. Comprueba la contraseña con `password_verify()`.
4. Si todo está bien, llena la mochila y te manda al panel (admin) o a los juegos (jugador). Si no, vuelta a `index.php?error=1`.

Las tres variables de sesión que crea son **las únicas que existen**. Apuntadlas tal cual:

VARIABLE	QUÉ GUARDA	EJEMPLO CON EL JUGADOR DE PRUEBA
`$_SESSION['id']`	El id del usuario en la tabla	2
`$_SESSION['nombre']`	Su nombre	Jugador Prueba
`$_SESSION['rol']`	admin o jugador	jugador

Fijaos en lo que **no** está: el saldo. Yo lo leo siempre de la base de datos.

El portero (sesion.php)

Es un bloque corto que va al principio de cada página protegida:

```
session_start();
if (!isset($_SESSION['id'])) {
    header("Location: index.php");
    exit;
}
```

Si en la mochila no hay `id`, no has hecho login, así que te devuelve al formulario.

La salida (logout.php)

`session_destroy()` vacía la mochila y te devuelve a `index.php`.

PREGUNTAS PARA PENSAR

- ¿Por qué no se guarda el saldo en `$_SESSION`? Pista: imagina que el admin te añade 50 € mientras estás jugando.
- El mensaje de error dice «Teléfono o contraseña incorrectos» y no «Ese teléfono no existe». ¿Qué ganamos con un mensaje tan poco concreto?
- ¿Qué es lo que hace el `?` de la consulta? ¿Qué podría pasar si pegamos el teléfono directamente dentro de la cadena SQL?

EJERCICIO 4.1 **Mira dentro de la mochila**

Cread un archivo de prueba en vuestra copia local que incluya `sesion.php` y muestre el contenido de `$_SESSION` con `var_dump`. Entrad con el jugador de prueba y abridlo. ¿Coinciden las claves con la tabla de arriba? Ahora cerrad sesión y volved a abrirlo. ¿Qué pasa?

EJERCICIO 4.2 **Predice antes de probar**

Sin hacer login, escribid en la barra de direcciones la URL de `juegos.php`. Antes de pulsar Intro, apuntad qué creéis que va a pasar y qué línea de qué archivo lo provoca. Haced lo mismo con `usuario.php`.

RETO 4.3 **¿Y si quito el exit?**

En `sesion.php`, justo después del `header()` hay un `exit`. Pensad (o probadlo en vuestra copia): si lo quitamos, ¿el servidor deja de ejecutar el resto de la página? ¿Podría llegar al navegador algo que no debería?

RETO 4.4 **La cookie que no cambia**

Abrid las herramientas del navegador, pestaña Aplicación > Cookies, y apuntad el valor de `PHPSESSID` antes de hacer login. Haced login y volved a mirarlo. ¿Ha cambiado? Buscad en `doc_tecnica/03-autenticacion-sesiones.md` la función `session_regenerate_id` y averiguad para qué sirve. ¿Está en `usuario.php`?

RETO 4.5 **Nombres que no existen**

La documentación de `doc_tecnica` habla de una variable `$_SESSION['usuario']`. Buscadla en el código. Si vuestro juego la usara, ¿qué mostraría?

> *Conclusión para apuntar:* ***los nombres se copian del código****, no de la memoria ni de la documentación antigua.*

ENTREGA — la salida del `var_dump` (antes y después de cerrar sesión), vuestras predicciones de 4.2 junto a lo que pasó de verdad y la respuesta a los tres retos.

CA.5 La landing: juegos.php

ORDENADOR · PAREJA · 50 MIN

`juegos.php` es la página que ve el jugador al entrar: «Bienvenido *nombre*», «Tu saldo es *X* €» y una tarjeta por juego. Vuestro juego tendrá que empezar exactamente igual que ella, así que es el mejor modelo que tenéis. Cómo funciona, de arriba abajo:

1. `require "sesion.php";` → si no hay login, fuera.
2. `require "conexion.php";` → ya tenemos `$conexion`.
3. Consulta preparada: `SELECT nombre, saldo FROM usuarios WHERE id = ?`, con el `?` sustituido por `$_SESSION['id']`.
4. El resultado queda en un array llamado **`$usuario`**: `$usuario['nombre']` y `$usuario['saldo']`.
5. En el HTML, el nombre se pinta con `htmlspecialchars()` y el saldo con `number_format($usuario['saldo'], 2, ',', '.')`, que lo convierte en un texto como `1.234,50`.
6. Si el rol es admin, aparece también el enlace al panel.

Los nombres de archivo que espera la landing

Cada tarjeta enlaza con un archivo concreto. Si vuestro juego no se llama así, el enlace dará un error 404.

JUEGO	ARCHIVO QUE ENLAZA JUEGOS.PHP
Cara o cruz	`caraocruz.php`
La ruleta	`ruleta.php`
Piedra, papel o tijera	`piedrapapeltijera.php`
Tragaperras	`tragaperras.php`
Dados, blackjack, pares o nones…	Todavía no tienen tarjeta: la añadiremos en clase

PREGUNTA PARA PENSAR

El saldo se pide a la base de datos cada vez que se carga la landing, aunque acabes de verlo. ¿No sería más rápido guardarlo? ¿Qué ganamos con preguntarlo siempre?

EJERCICIO 5.1 **Tu tarjeta**

En vuestra copia local, copiad uno de los bloques `<a class="juego" ...>` y cread la tarjeta de vuestro juego. Fijaos en las cuatro partes: el `href`, el icono, el `h3` y la descripción. ¿Qué nombre de archivo vais a usar? Usad minúsculas, sin espacios ni tildes. ¿Por qué esa norma?

EJERCICIO 5.2 **Cambia el saldo desde fuera**

Con la landing abierta, cambiad el saldo del jugador de prueba en phpMyAdmin y recargad. ¿Se actualiza? Explicad por qué con lo que habéis visto en CA.4.

RETO 5.3 **El saldo que se encoge**

Un compañero propone: «Para pasar el saldo a mi juego, cojo con JavaScript el texto del `<strong>` de la landing y le hago `parseFloat()`». Probadlo en la consola con el texto `"1.234,50"`. ¿Qué número sale? ¿Qué conclusión sacáis sobre usar texto «bonito» para hacer cálculos?

RETO 5.4 **Un nombre peligroso**

Desde el panel de control, cread un usuario cuyo nombre sea `<b>Hola</b>` y entrad con él. ¿Cómo se ve el saludo? ¿Qué pasaría si quitáramos `htmlspecialchars()`? ¿Y si en vez de `<b>` alguien escribiera una etiqueta `<script>`?

RETO 5.5 **El jugador fantasma**

Abrid dos navegadores: en uno, entrad como jugador; en el otro, como admin. Desde el panel, borrad a ese jugador. Ahora recargad la landing del jugador. ¿Qué aparece? ¿Por qué `sesion.php` no lo ha echado si el usuario ya no existe? ¿Qué comprobación faltaría?

ENTREGA — el bloque HTML de vuestra tarjeta, la explicación de 5.2 y la respuesta a los tres retos.

CA.6 El panel de control

ORDENADOR · PAREJA · 50 MIN

`panelcontrol.php` es una sola página que hace tres trabajos: dar de alta usuarios, añadir saldo y borrar. Para saber cuál le estás pidiendo, cada formulario lleva un campo oculto llamado **`accion`** con el valor `alta`, `saldo` o `borrar`. Todos los formularios se envían a la misma página, que procesa la acción y luego vuelve a pintar la tabla.

ACCIÓN	CAMPOS QUE ENVÍA EL FORMULARIO	QUÉ HACE EN LA BASE DE DATOS
`alta`	nombre, telefono, password, saldo, rol	INSERT con la contraseña pasada por `password_hash()`
`saldo`	id, cantidad	`UPDATE usuarios SET saldo = saldo + ? WHERE id = ?`
`borrar`	id	DELETE, salvo que intentes borrarte a ti mismo

Antes de todo eso, la página incluye `sesion.php` y además comprueba que `$_SESSION['rol']` sea `admin`. Si eres jugador, te manda a `juegos.php`.

Este panel es vuestra herramienta de pruebas: desde aquí os daréis saldo para probar vuestro juego, sin tocar el código.

PREGUNTAS PARA PENSAR

- En el alta, el rol no se guarda tal cual llega: `$_POST['rol'] == 'admin' ? 'admin' : 'jugador'`. Si el select del formulario solo tiene esas dos opciones, ¿por qué me molesto en comprobarlo otra vez en PHP?
- Para borrar hay un `confirm()` de JavaScript y, aparte, PHP comprueba que no te borres a ti mismo. ¿Por qué no basta con el `confirm()`?

EJERCICIO 6.1 **Tus jugadores de prueba**

Entrad como admin y cread tres jugadores: uno con 0 €, otro con 5 € y otro con 1000 €. Los usaréis en CA.7 para comprobar que vuestro juego se porta bien sin dinero, con poco dinero y con mucho.

RETO 6.2 **El admin generoso… o no**

En «Añadir saldo», escribid `-500` a un jugador que tenga 100 €. ¿Qué saldo le queda? Ahora escribid `abc`. ¿Qué mensaje sale? ¿Es verdad lo que dice? Apuntad qué validaciones faltan.

RETO 6.3 **Busca la diferencia**

El saldo inicial del alta y la cantidad de «Añadir saldo» se convierten a número de forma distinta. Cread un usuario con saldo inicial `10,50` y añadid `10,50` a otro. Comparad los resultados y encontrad la línea que marca la diferencia.

RETO 6.4 **El F5 que da dinero**

Añadid 10 € a un jugador y, justo después, pulsad F5 y aceptad el aviso del navegador. Mirad el saldo. ¿Qué ha pasado? Investigad qué es el patrón *Post/Redirect/Get* y por qué lo evitaría.

RETO 6.5 **El mensaje que miente**

Cread un usuario con un teléfono que ya existe: sale «Error: ese teléfono ya existe». Ahora probad con un teléfono nuevo pero con un nombre de más de 100 caracteres. ¿Qué mensaje sale? ¿Es el error de verdad? ¿Qué tiene de peligroso un `catch` que da siempre la misma explicación?

ENTREGA — captura del panel con vuestros tres jugadores de prueba y, para cada reto, qué hicisteis, qué salió y qué cambio lo arreglaría (por escrito, sin tocar mi rama).

CA.7 Integrar tu juego

ORDENADOR · PAREJA · 2 SESIONES + DISEÑO DE LA FASE 2 EN CLASE

Vuestro juego pasa de ser un `.html` suelto a ser una página `.php` del casino. La lógica en JavaScript se queda casi igual; lo que cambia es **de dónde sale el saldo** (de la base de datos, no de una variable inventada) y **quién puede entrar** (solo quien ha hecho login).

Lo hacemos en dos fases. La fase 1 ya se puede hacer con mi código. La fase 2 (guardar el saldo después de jugar) todavía **no existe**: la diseñaremos juntos en clase.

Fase 1 · Tu juego dentro del casino

1. Renombra tu página con el nombre que espera la landing (CA.5), por ejemplo `ruleta.php`, y déjala en la misma carpeta que `juegos.php`.
2. Las primeras líneas, antes de cualquier HTML, son las mismas que en `juegos.php`: incluir `sesion.php` y `conexion.php`, y pedir nombre y saldo del usuario con `$_SESSION['id']`.
3. Pasa el saldo a JavaScript **como número**, no como texto formateado. Dos formas típicas: un atributo `data-saldo` en un elemento del HTML que luego lees desde JS, o una constante en un `<script>` pequeño antes de cargar tu `.js`.
4. Tu archivo `.js` sigue aparte y se carga con una ruta relativa (con `/`, nunca con `\`).
5. Añade un enlace para volver a `juegos.php` y otro a `logout.php`, como en la landing.

El esqueleto, sin rellenar, está en `Aplica/act-CA.7-integrar-tu-juego/plantilla/mijuego.php` (los huecos son vuestros):

```
<?php
require "sesion.php";
require "conexion.php";
// 1. La misma consulta que juegos.php: nombre y saldo del usuario de la sesión
// 2. Guarda el resultado en $usuario
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Casino DAW - (tu juego)</title>
</head>
<body>
    <!-- 3. Saludo con el nombre (¡con htmlspecialchars!) -->
    <!-- 4. El saldo, que tu JS pueda leer como número -->
    <!-- 5. Tu juego -->
    <!-- 6. Enlaces a juegos.php y logout.php -->
    <script src="js/(tu-juego).js"></script>
</body>
</html>
```

Fase 2 · Guardar el saldo (por diseñar)

Cuando el jugador gane o pierda, la base de datos tiene que enterarse. En `doc_tecnica/arquitectura` dejé apuntada la idea de un archivo común, `juegos/api/jugar.php`, al que el JavaScript llamaría con `fetch()`. **Ese archivo no existe todavía**, ni están decididos sus nombres de campos. No lo inventéis por vuestra cuenta: lo definiremos en clase y lo añadiremos a esta guía.

Mientras tanto, preparad vuestro código para que el cambio sea fácil:

- Una función que **decide** el resultado (gana, pierde, cuánto) sin tocar la pantalla.
- Otra que **pinta** el resultado.
- Un único sitio donde se cambia el saldo. Ahí es donde irá la llamada al servidor.

PREGUNTAS PARA PENSAR

- Si abrís `ruleta.php` con doble clic desde el explorador de archivos, ¿se ejecuta el PHP? ¿Qué diferencia hay entre `file:///...` y `http://localhost/...`?
- ¿Dónde es más seguro decidir si el jugador ha ganado: en su navegador o en el servidor?

RETO 7.1 Hackea tu propio casino

Con vuestro juego ya integrado, abrid la consola y cambiad el valor de vuestra variable de saldo a 999999. Jugad. ¿Os deja? ¿Ha cambiado algo en la base de datos? Ahora imaginad que, en la fase 2, el JavaScript le dice al servidor «mi saldo nuevo es 999999» y el servidor se lo cree. Debatid: ¿qué debería enviar el navegador y qué debería calcular el servidor?

RETO 7.2 Dos pestañas, un saldo

Abrid vuestro juego en dos pestañas a la vez y jugad en una. ¿Qué saldo muestra la otra? Si las dos guardaran el saldo en la base de datos, ¿cuál ganaría? ¿Cómo lo evita `saldo = saldo + ?` del panel?

RETO 7.3 ¿Cuándo guardo?

Comparad dos ideas: guardar el saldo después de cada jugada o guardarlo solo al pulsar «Salir». Para cada una, apuntad una ventaja y un problema. Pista: ¿qué pasa si el jugador cierra la pestaña sin pulsar nada?

RETO 7.4 La carpeta equivocada

La arquitectura de `doc_tecnica` pone los juegos en una carpeta `juegos/`, pero `juegos.php` enlaza con `ruleta.php` en la raíz. Si movierais vuestro juego a `juegos/ruleta.php`, ¿qué tres cosas dejarían de funcionar? Pensad en los `require`, en el enlace de la landing y en el `header("Location: index.php")` de `sesion.php`.

CRITERIO IA

Si le pedís a una IA que «integre mi juego en el casino», lo más probable es que os devuelva un código limpio… con nombres inventados (`$pdo`, `$_SESSION['usuario']`, `login.php`) y el resultado decidido en JavaScript. Antes de pegar nada: ¿tiene sentido pedirle esto sin enseñarle mi código? ¿Qué contexto necesitaría (pista: la chuleta entera)? ¿Qué líneas no le confiaríais sin revisión? Todas las que tocan el saldo.

Lista de comprobación de la fase 1

- ☐ Mi juego se llama como espera la landing
- ☐ Sin login, me devuelve a `index.php`
- ☐ Aparece mi nombre y el saldo real de la base de datos
- ☐ El saldo llega a JS como número (lo he comprobado con `typeof` en la consola)
- ☐ Con 0 € no puedo apostar, y la apuesta vacía, negativa o mayor que el saldo no se acepta
- ☐ La lógica y la pantalla están en funciones separadas
- ☐ Mi `conexion.php` con datos reales no está en mi commit

ENTREGA — commit en vuestra rama con el juego integrado, la lista de comprobación marcada en el README y la respuesta escrita a los cuatro retos.

Chuleta de nombres

Todo lo que vuestro juego puede usar, tal y como lo he escrito en mi rama `Alberto`. **Si un nombre no está aquí, no existe.**

QUÉ	NOMBRE EXACTO	DÓNDE SE CREA
Base de datos	`casino`	casino.sql
Tabla	`usuarios`	casino.sql
Columnas	`id`, `nombre`, `telefono`, `password`, `saldo`, `rol`	casino.sql
Valores de rol	`admin`, `jugador`	casino.sql
Variable de conexión	`$conexion`	conexion.php
Variables de sesión	`$_SESSION['id']`, `$_SESSION['nombre']`, `$_SESSION['rol']`	usuario.php
Datos del usuario en la landing	`$usuario['nombre']`, `$usuario['saldo']`	juegos.php
Campos del login	`telefono`, `password`	index.php
Error de login	`index.php?error=1`	usuario.php
Proteger una página	`require "sesion.php";`	sesion.php
Archivos de juego	`caraocruz.php`, `ruleta.php`, `piedrapapeltijera.php`, `tragaperras.php`	juegos.php
Volver / salir	`juegos.php`, `logout.php`	juegos.php
Acciones del panel	`alta`, `saldo`, `borrar` (campo oculto `accion`)	panelcontrol.php
Usuarios de prueba	600000000 / admin123 (admin) 611111111 / 1234 (jugador)	casino.sql

La regla de oro, otra vez

No inventéis nombres. Si vuestro código usa un nombre que no está en esta chuleta, no es vuestro juego el que está mal: es que todavía no encaja. Y lo que no encaja hoy, mañana, en una empresa, es una incidencia en producción.